



Setting Up Full Multigrid

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, Tricks of the Trade, development

Most of the effort in a geodynamics model goes into solving the Stokes equations, and most of that into the velocity solver. How that solver is preconditioned sets how long a model takes to run. Underworld can build the preconditioner from a real hierarchy of meshes rather than from the matrix alone, and this note is about how to get that hierarchy and what it is worth.

1. WHAT MULTIGRID DOES

An iterative solver reduces error unevenly. Simple relaxation methods (smoothers) are good at removing error that varies rapidly from cell to cell, and poor at removing error that varies smoothly across the whole domain. The smooth part is then what costs: it takes many sweeps to carry information from one side of a fine mesh to the other.

Multigrid turns that around. Error that is smooth on a fine mesh is *not* smooth on a mesh twice as coarse, where it spans half as many cells, so a few sweeps there remove what the fine mesh could not.

The method is a recursion: smooth on the fine mesh, transfer what is left to a coarser one, solve there with another recursive step, transfer the correction back, smooth again. Each *level* handles the band of error it can see cheaply, and the work per unknown stops growing as the model gets bigger.

Two things have to be supplied: the coarse levels, and the operators that move between them.

2. TWO WAYS TO BUILD THE COARSE LEVELS

Algebraic multigrid (AMG) builds the coarse levels from the matrix. It inspects the operator's connectivity, groups unknowns that are strongly coupled, and constructs coarse levels and transfer operators from that grouping alone. Because it asks nothing of the mesh it works everywhere, which makes it the sensible default. PETSc's implementation is GAMG, and it is what Underworld uses when it has nothing better.

Geometric multigrid (GMG) uses actual coarser meshes. If the fine mesh was built by *refining* a coarse one, those coarser meshes already exist, and the transfers follow from the refinement relation: each fine node either coincides with a coarse node or lies inside a known coarse cell. In PETSc this is `pc_type=mg`, and Underworld runs it as a *full* multigrid cycle: solve on the coarsest mesh, interpolate that solution up to the next mesh

as a starting point, and repeat, cycling up and down through the levels at each stage to clear the error the interpolation leaves behind. We'll refer to this as FMG.

The difference between them shows up when the operator's connectivity is a poor guide to the geometry, which is what jumps and steep gradients in material properties produce. The algebraic method sees a matrix whose couplings are dominated by the stiff region and groups unknowns accordingly; the geometric method uses the grids it was given and is indifferent to the coefficients. Which of those is the better bet is not obvious in advance, so it is measured below. Switching the geometric solver on comes first, and that starts with the mesh.

3. MAKING A MESH WITH A HIERARCHY

A mesh only carries a hierarchy if it was built with one. That is the `refinement` argument:

```
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)
len(mesh.dm_hierarchy)      # 3: the base and two refinements
```

The mesh you get back is the finest level, and the coarser ones are stored within it as PETSc `DMPlex` objects in `mesh.dm_hierarchy`. Those are what the preconditioner uses. They are not Underworld meshes — there is no `Mesh` object for a coarse level, and making one takes work — so the hierarchy is something the solver reads rather than something you handle.

Build the same mesh at `cellSize=0.0125` with no refinement and you get about the same resolution with no hierarchy at all, and so no geometric multigrid. Algebraic multigrid runs perfectly well on a mesh that has a hierarchy; it just does not use it.

The solver picks it up on its own:

```
stokes = uw.systems.Stokes(mesh)
stokes.solve()      # preconditioner defaults to "auto"
```

"auto" uses geometric multigrid when `len(mesh.dm_hierarchy) > 1` and GAMG when it does not. You can also ask directly, with `"fmg"` or `"gamg"`. Two behaviours are deliberate: "auto" only ever adds geometric multigrid to an untouched configuration, so it will not overwrite preconditioner options you have set yourself; and where a request cannot be honoured, you can find the reasoning in `stokes.pc_fallbacks`. `stokes.preconditioner_settings` reports what was actually applied.

4. CURVED BOUNDARIES NEED HELP TO REFINE

Refining a mesh puts new nodes at the midpoints of existing edges. On a straight boundary that is fine. On a curved one it is not: the coarse mesh approximates a circle by a polygon, and the midpoint of a chord lies inside the circle rather than on it. Those midpoints have to be moved back onto the boundary each time the mesh is refined.

Underworld's curved meshes carry a refinement *callback* that snaps boundary-labelled nodes back onto the true surface after each refinement. For the annulus it is a few lines — take the nodes labelled `Upper` and `Lower` and rescale each to the correct radius:

```
coords[upper] *= radiusOuter / R[upper]
coords[lower] *= radiusInner / R[lower]
```

This happens automatically for the built-in meshes, so in normal use there is nothing to do. It matters if you build a mesh of your own with curved boundaries: without a callback of this kind, the finest mesh still has the coarsest mesh's faceting, and the boundary it represents is the polygon rather than the circle.

5. WHAT THE COARSE MESH LEAVES BEHIND

Because the fine mesh is made by subdividing the coarse one, the coarse mesh's own triangulation is still visible in it. Its edges survive as continuous lines across the fine mesh, and its vertices remain places where an unusual number of elements meet. The pattern is a record of how the mesh was made rather than of the problem being solved.

`mesh.relax()` moves the nodes to improve element shapes while keeping the resolution and the topology, which loosens that imprint.

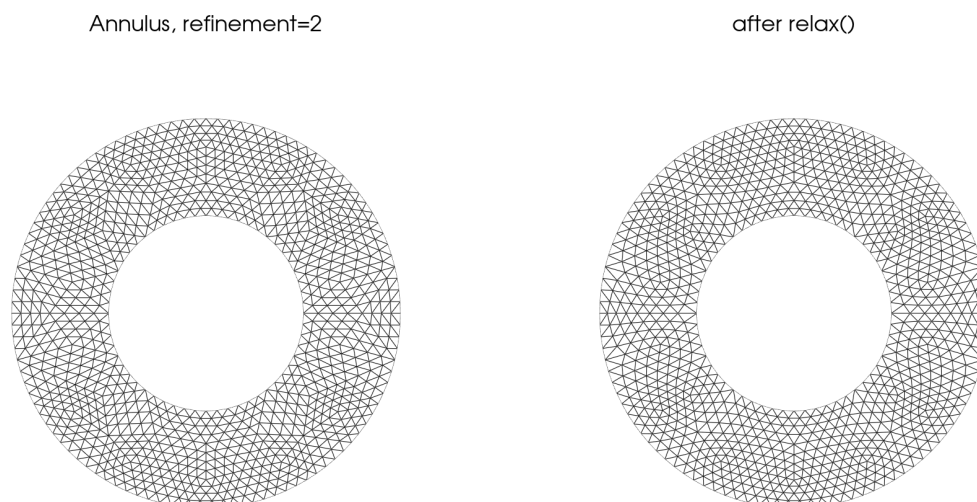


Figure 1: A twice-refined annulus, before and after `relax()`. The seams on the left are the coarse mesh's own edges, still visible after two refinements. Median element quality goes from 0.940 to 0.972 and the tenth percentile from 0.839 to 0.927; the worst single cell goes the other way, 0.827 to 0.763, because the mover minimises a global energy and will trade a few cells for many. Every node that began on a bounding circle is still exactly on it: relaxation moves interior coordinates only.

Relaxation moves coordinates but not topology, so the refinement relation between the levels is untouched and the hierarchy survives it. Moving a mesh without rebuilding it is the subject of a [related technical note](#).

6. WHEN THE CHOICE MATTERS

The test is SolKz, a standard Stokes benchmark on the unit box: viscosity varying exponentially with depth, $\eta = e^{2Bz}$, forced by $\mathbf{f} = (0, \sin(m\pi z) \cos(n\pi x))$ with $n = 3, m = 2$, free slip on all four walls, Taylor–Hood P_2 – P_1 elements. The viscosity contrast across the box is e^{2B} . SolKz has a closed-form solution, but nothing here uses it: these are timings, not accuracy measurements.

Work is reported as floating-point operations counted by PETSc, per unknown and relative to the cheapest FMG run. Flops are the better measure: they do not depend on the machine, and they are deterministic — repeating a run reproduces the count exactly, where a timing moves by a few per cent. Times are given alongside, because they are what you wait for and they do not tell the same story.

Both preconditioners are run at PETSc’s own defaults, and GAMG is told that the velocity field has two components per node so that it aggregates nodes rather than scalars.

Cost against viscosity contrast, at 11 727 unknowns:

preconditioner	viscosity contrast	relative work per unknown	relative time per unknown	cycles per velocity solve
FMG	10^0	1.0	1.0	3
FMG	10^2	1.3	1.1	3
FMG	10^4	1.6	1.2	3
FMG	10^6	1.8	1.3	3
GAMG	10^0	2.2	1.2	34–47
GAMG	10^2	3.3	1.5	39–49
GAMG	10^4	4.0	1.7	36–48
GAMG	10^6	4.0	1.6	30–44

Both handle the whole range, and neither degrades relative to the other as the viscosity gets harder. GAMG does a little over twice FMG’s arithmetic — the ratio runs 2.2, 2.5, 2.5, 2.2 — and FMG holds three cycles per velocity solve throughout where GAMG needs thirty to fifty.

Cost against problem size, at constant viscosity:

preconditioner	unknowns	relative work per unknown	relative time per unknown	Gflop/s
FMG	2 947	1.0	1.0	2.2
FMG	11 727	1.9	1.3	3.3
FMG	46 783	2.6	1.6	3.7
FMG	186 879	2.9	1.9	3.4
FMG	747 007	3.0	2.3	3.0
GAMG	2 947	3.3	1.5	4.9
GAMG	11 727	4.7	1.8	5.7
GAMG	46 783	4.9	2.0	5.6
GAMG	186 879	5.8	2.2	5.8

Both flatten. Step by step, FMG’s flop count goes as $N^{1.46}$, $N^{1.23}$, $N^{1.08}$, $N^{1.03}$; GAMG’s as $N^{1.25}$, $N^{1.04}$, $N^{1.11}$. Both are converging on linear, which is what multigrid of either kind is supposed to do: the algebraic method reaches it as surely as the geometric one.

The cycle counts say the same thing. FMG converges the velocity block in three cycles at every size; GAMG takes 26–32, then 34–47, then 33–49 — ten times as many, but not a growing number. (The Schur factorisation invokes the velocity solve sixteen times per Stokes solve, the same for both, so that factor cancels.)

So the geometric hierarchy is worth roughly a factor of two: FMG does about half GAMG’s arithmetic on this problem, and holds that through six orders of viscosity contrast and a 250-fold growth in problem size.

In wall clock the advantage is smaller still — 2.3 against 2.2 at the largest size, which is no advantage at all — and the last column says why. GAMG runs at 5–6 Gflop/s where FMG runs at 2–4, and FMG’s rate *falls* at the largest size as the problem outgrows cache.

The difference is in the shape of the arithmetic rather than the amount of it. Geometric multigrid on an unstructured hierarchy works through sparse, indirect operators, and its coarse levels are small enough that they cannot keep the processor busy. Algebraic multigrid does more arithmetic, but in denser and more regular operators that run closer to the machine’s peak rate. On this hardware the two effects very nearly cancel. The flop count is a property of the method and should carry to another machine; the rate is a property of this one, and on a machine with a different memory system the wall-clock comparison could come out either way.

What this comparison does not settle

The solver tolerance is held fixed as the mesh refines. That is the usual way to show a multigrid scaling, and it is a choice: a finer mesh has a smaller discretisation error, so an argument can be made for tightening the solver tolerance alongside it, which would make the work per unknown grow. The table answers “what does it cost to solve this system”, not “what does it cost to reach the accuracy the mesh can support”.

SolKz is also a smooth problem. Viscosity that varies smoothly everywhere is what algebraic coarsening reads best; structure that is concentrated rather than distributed is harder for it, and that case is not measured here.

7. USING IT

```
# Build the hierarchy at mesh construction: base + two refinements.
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)

stokes = uw.systems.Stokes(mesh)
stokes.preconditioner = "auto"      # FMG when a hierarchy exists
stokes.solve()

print(len(mesh.dm_hierarchy))      # how many levels there are
print(stokes.preconditioner_settings) # what was applied
print(stokes.pc_fallbacks)         # anything declined, and why
```

If a solve is slow and the mesh has no hierarchy, this is the first thing to change: rebuild it with `refinement` rather than at a fine `cellSize`, and the same resolution arrives with the coarse levels attached. Note that `cellSize` then sets the *coarsest* mesh, and each refinement halves it — so `cellSize=0.05` with `refinement=2` resolves like `cellSize=0.0125`.